

Welcome to CS 225: “Data Structures”Course Website: <https://learn.intl.zju.edu.cn>

Description: Data abstractions: elementary data structures (lists, stacks, queues, and trees) and their implementation using an object-oriented programming language. Solutions to a variety of computational problems such as search on graphs and trees. Elementary analysis of algorithms.

Instructors:**Volodymyr Kindratenko** <kindrv@intl.>, Prof.**Yushi Cheng** <yushicheng@intl.zju.edu.cn>, Prof.**Gaoang Wang** <gaoangwang@intl.zju.edu.cn>, Prof.**Wendao Yao** <wendao.22@intl.zju.edu.cn>**Kerui Zhang** <kerui.23@intl.zju.edu.cn>**Qi Li** <qil.23@intl.zju.edu.cn>**Licheng Xu** <licheng.22@intl.zju.edu.cn>**Yuxuan She** <yuxuan1.23@intl.zju.edu.cn>**Jincheng An** <jincheng.23@intl.zju.edu.cn>

Lab Sections: As part of registering for CS 225, you have also registered for a lab section. **Labs start this week** and are held by your TA – your lab TA is the first person who you should contact if you’re having any troubles with the course!

Coursework and Grading

A total of 1,000 points are available in CS 225, along with many opportunities to earn extra credit. The points are broken down in the following way:

- **100 points:** Weekly lab assignments
 - There are 14 weeks -- points above 100 are extra credit!
- **130 points:** MPs (10 for MP1, 20 for MP2-7)
 - Early submissions of MPs earns up to +7 points /MP after MP1
- **520 points** (6 x 80 + 1 x 40 pts): Bi-weekly quizzes
 - short answer exams (theory/math-focused)
 - programming exams (coding focused)
- **250 points:** Final Exam

Usual Final Course Grades

Your final course grade is determined by the number of points you earned during the semester:

Points	Grade	Points	Grade	Points	Grade
[970, ∞)	A+	[930, 970)	A	[900, 930)	A-
[870, 900)	B+	[830, 870)	B	[800, 830)	B-
[770, 800)	C+	[730, 770)	C	[700, 730)	C-
[670, 700)	D+	[630, 670)	D	[600, 630)	D-
		(600, 0]	F		

We never curve individual exam or assignment scores; instead, if necessary, we may lower the points required for each grade cutoff to be lower than the stated cutoff. However, this has not been necessary in recent semesters. In no case will we raise the cutoff.

Contacting Us:

We want to do everything we can to make sure you have a great time learning data structures. There’s a lot of ways to reach out if you get stuck:

1. **Course Piazza:** Online Q&A discussion forum to ask and answers from your peers and course staff. Join it!
2. **Open Office Hours:** TAs will be holding office hours every week, starting from this week. Great way to get help on MPs, labs, and technical content.
 - Location:** Check CS 225’s Blackboard for place
 - Times:** Check CS 225’s Blackboard for time
3. **Lab TA:** Your Lab TA is your primary administrative contact for the course. You will see him each week in lab and can reach him via e-mail. However, no coding assistance will be provided via e-mail (use office hours).

Variables and Classes in C++

Every variable is defined by four properties:

- 1.
- 2.
- 3.
- 4.

And every variable is one of two types:

<code>int myFavoriteInt;</code> <code>char grade = 'A';</code> <code>double gamma = 0.653;</code>	<code>Sphere myFavoriteSphere;</code> <code>Cube rubix;</code> <code>Grade courseGrade;</code>

Creating New Types

In data structures, we will be learning and creating new types of structures to store data. We will start simply – by the end, we will have types we built being the building blocks for new types!

Big Idea: Encapsulation

Encapsulation principles:

sphere.h		sphere.cpp	
1		1	
2		2	
3		3	
4		4	
5		5	

Our first class:

sphere.h		sphere.cpp	
1	<code>#ifndef SPHERE_H</code>	1	<code>#include "sphere.h"</code>
2	<code>#define SPHERE_H</code>	2	
3		3	<code>double</code>
4	<code>class Sphere {</code>	4	<code> Sphere::getRadius() {</code>
5	<code> public:</code>	5	
6		6	<code> }</code>
7		7	
8		8	
9		9	
10		10	
11		11	
12	<code> private:</code>	12	
13		13	
14		14	
15	<code>};</code>	15	
16		16	
17	<code>#endif</code>	16	

...run this yourself: run `g++ sphere.cpp -o sphere` in the lecture source code.

Key Concepts in C++ Classes:

Every class will be made up of common key pieces:

1. _____
`sphere.h:1 :2 and :17`
2. _____
`sphere.h:4..15`
3. _____
`sphere.cpp:1`
4. _____
`sphere.cpp:3`

Public vs. Private:

Situation	Protection Level
Helper function used internally in <code>Sphere</code>	
Variable containing data about the <code>Sphere</code>	
<code>Sphere</code> functionality provided to client code	

Hierarchy in C++:

There **sphere** class we're building might not be the only **sphere** class. Large libraries in C++ are organized into _____.

sphere.h		sphere.cpp	
1	#ifndef SPHERE_H	1	#include "sphere.h"
2	#define SPHERE_H	2	
3		3	namespace cs225 {
4	namespace cs225 {	4	double
5	class Sphere {		Sphere::getRadius() {
6	public:	5	return r_;
7	double getRadius();	6	}
...	/* ... */	7	}

Our first Program:

main.cpp	
1	#include "sphere.h"
2	#include <iostream>
3	
4	int main() {
5	cs225::Sphere s;
6	std::cout << "Radius: " << s.getRadius() << std::endl;
7	return 0;
8	}

Several things about C++ are revealed by our first program:

5.

main.cpp:4
6.

main.cpp:5, main.cpp:1
7.

main.cpp:6, main.cpp:2

Simplify the Syntax

Often, we will find ourselves using significant functionality from a single library and typing `cs225::` or `std::` becomes burdensome. We can import an entire namespace into the global scope with:

main.cpp	
1	#include "sphere.h"
2	#include <iostream>
3	
4	using namespace std;
5	using namespace cs225;
6	
7	int main() {
8	Sphere s;
9	cout << "Radius: " << s.getRadius() << endl;
10	return 0;
11	}

...run this yourself: run `g++ main.cpp sphere.cpp -o sphere; ./sphere` in the lecture source code.

Big Idea: Constructor

Default Constructor:

Every class in C++ has a constructor – even if you didn't define one!

- Automatic Default Constructor:
- Custom Default Constructor:

sphere.h		sphere.cpp	
...		...	
4	class Sphere {	3	Sphere::Sphere() {
5	public:	4	
6	Sphere();	5	
...	/* ... */	6	}
		...	

Custom Constructors:

We can provide also create constructors that require parameters when initializing the variable:

sphere.h		sphere.cpp	
...		...	
4	class Sphere {	3	Sphere::Sphere(double r) {
5	public:	4	
6	Sphere(double r);	5	
...	/* ... */	6	}
		...	

INSIGHT PUZZLE – What happens when we run main.cpp?

main.cpp w/ above custom constructor	
...	
8	Sphere s;
9	cout << "Radius: " << s.getRadius() << endl;
...	

INSIGHT PUZZLE – How do we fix it?

1. _____
2. _____

Pointers – Introduction

Besides classes, the other major component of C++ that will be used throughout all of CS 225 is the use of pointers. Pointers are:

- Extremely power, but extremely dangerous
- A **level of indirection** via memory to the data.

As a level of indirection via memory to the data:

1. _____
2. _____

The addition of a (*) to the end of the type denotes it a pointer:

```
Sphere s1; // A variable of type Sphere
Sphere *s2; // A variable of type Sphere pointer
```

With a pointer type, members are accessed via the arrow operator (->) instead of the dot operator (.):

```
Sphere s1; // A variable of type Sphere
cout << s1.getRadius() << endl;

Sphere *s2 = &s1; // A variable of type Sphere pointer
cout << s2->getRadius() << endl;
```

CS 225 – Things To Be Doing:

1. Lab Sections –Thursday
2. MP1 will be released soon, due Monday, March 2
3. Visit the course Blackboard often!
4. Post questions on Piazza